

`self.nextAccountNumber` ❶. The prefix `self.` designates these as instance variables, meaning the `Bank` class can refer to these variables in any of its methods.

Second, there are two methods for creating an account: `createAccount()` and `openAccount()`. The `createAccount()` method instantiates a new account ❷ with the user's name, a starting amount, and a password passed in for the new account. The `openAccount()` method asks the user questions to obtain these three pieces of information ❸ and calls the `createAccount()` method within the same class.

Having one method call another method in the same class is common. But the called method doesn't know whether it was called from inside or outside the class; it only knows that the first argument is the object on which it should run. Therefore, the call to the method must start with `self.`, because `self` always refers to the current object. Generically, to call from one method to another method in the same class we need to write:

```
def myMethod(self, <other optional parameters>):  
    ...  
    self.methodInSameClass(<any needed arguments>)
```

After collecting information from the user for `openAccount()`, we have this line ❹:

```
userAccountNumber = self.createAccount(userName, userStartingAmount,  
userPassword)
```

Here, `openAccount()` calls `createAccount()` from the same class to create the account. The `createAccount()` method runs, instantiates an `Account` object, and returns an account number to `openAccount()`, which returns that account number back to the user.

Finally, the new method `closeAccount()` allows the user to close an existing account ❺. This is an extra piece of functionality we'll offer from our main code.

Our `Bank` class represents an abstract view of a bank rather than the physical brick-and-mortar object. This is another good example of a class that does not represent a physical structure.